

Discrete Event Simulation Analysis of a Web Application

Assignment is to be done by TWO team members.

Important Deadlines

References in [Dropbox](#) folder

1. ~~Simulation Program Functional Demo (Code submission): Tuesday, March 3rd 2026~~

- **Submit all your main code and scripts, and input files if any.**
- **Code should be well-commented**
- **Code should produce a clean, somewhat readable trace.**
- **In your demo you will have to point out (reason-through) a few examples (from trace output) of why you think the code is correct**
- **You need not show metrics calculation at this point.**

2. *Final Submission (Code, Analysis in slides format) due: ~~Tuesday~~, Wednesday March 11th 40th, 2026:*

Submit a file called `assmt2_final_name1_name2.tar`. Tarball should have:

- *A PDF of a report in the form of *SLIDES*. Please do not write a "document". SLIDES ONLY. The slides should describe your system assumptions, your simulation class design, highlights of the code, e.g. the event handlers etc, the post-processing, and finally the results in the form of graphs and inferences. Results should start with QUESTIONS asked and then experiments done to answer them. At least ONE metric *MUST* have confidence intervals.*
- *Simulation Source code*
- *Any post-processing scripts code*
- *Sample input data files.*
- *Spreadsheets if any*
- *README explaining the content of the tar and how to run the code and generate results/graphs etc and with any links in case sheets are on google*

Overall Goal of the Assignment

In this assignment you will continue the study of the performance of a Web Application, this time through a discrete event simulation model. You will take advantage of the power of simulation analysis and model behaviours that are difficult to capture using theoretical queuing systems. You will write a program, and then run it to ask various interesting questions. For all metrics you will perform proper statistical soundness analysis.

Detailed Specifications

Write a simulation program in any general purpose simulation language (except C. It is not a good habit in today's times to program in C - please use every opportunity to program in object oriented languages. C++, Java or python is fine.)

System Characteristics

Your system should have the characteristics of the Web server environment that you measured. The following is **recommended, you can add/modify.**

- Multi-core server machine
 - *You can assume thread-to-core affinity. Once a thread is “assigned” to a core, it will remain there*
- Multi-threaded Web server
- Thread-per task model - until max is reached, after which queuing for threads starts.
- Round-robin scheduling, **with context-switching overhead**
- Request time-outs (and perhaps retries)
- Users are in a standard closed loop - issue request, wait for response, think then issue request again.
 - *Think time should have a mode not close to zero - don't assume exponential think time*
- Have options of various request execution time distributions, such as - constant, uniform, exponential.
- Request timeout should also have a well-chosen distribution
 - *Have some sort of a minimum, and then a variable component*

Performance Metrics

Metrics/graphs will be similar to what you measured for assignment. One set of your graphs should be “measured vs MVA model vs simulation model” comparison charts, which you should get by using the parameters of your measured system.

- Average Response Time vs number of users
 - Generate confidence intervals for this metric. Point estimates (averages of estimates from independent runs) are ok for the remaining metrics

- Throughput, Goodput, Badput vs Number of users
 - Badput = rate of completion of requests which were already timed out.
 - Goodput = rate of completion of requests that were not timed out
 - Throughput = goodput + badput
- Request Drop rate vs number of users
- Average core utilization vs Number of users
- Some additional graphs representing your own curiosity regarding system performance vs some system parameter

Ensure that the transient is determined and discarded in each of your simulation runs. You can do this informally, or follow Welch's procedure.

Note that the modeling and analysis expected in this Assignment is open-ended, and simply plotting the charts and simulating behaviours suggested above may not guarantee 'full marks'. You can ask various other analysis questions, even code different architectures of Web servers, various scheduling policies, overload control etc. Or ask good "What-If" questions from the above set up.